CHAPTER 1

Floating Point Arithmetic

1.1 Why Computers Can't Count Perfectly

Computers are known to be very precise. Modern processors can perform billions of calculations per second, so it's not intuitive to realize they have this persistent limitation: they cannot store most numbers exactly.

Let us think about what it would actually take to store a number like $\frac{1}{3}$. You probably know its decimal expansion:

$$\frac{1}{3} = 0.3333333\ldots$$

The digits go on forever. Now imagine you are a computer with a fixed amount of memory. You cannot store infinitely many digits. So you store 0.333 or 0.3333333, depending on how much space you have, and you accept that the result is not exactly right. That gap between the number you wanted and the number you actually stored is called *approximation error*, and it never fully goes away.

This is not just a problem for fractions with repeating decimals. Irrational numbers like $\sqrt{2} = 1.41421356\ldots$ and transcendental numbers like $\pi = 3.14159265\ldots$ also have infinite, non-repeating decimal expansions. Every one of them must be approximated when stored in a computer.

To handle this systematically, mathematicians and engineers developed the *floating-point system*. Before we look at how it works, it helps to understand the two core reasons why any finite system will always fall short of the real number line.

1.1.1 The Number Line is Continuous and Unbounded

Between any two real numbers, no matter how close together, there are infinitely many others. Pick any two decimals you like. For example, if you picked 1.00 and 1.01, you can always find a number between them: 1.005, 1.0051, 1.00500001, and so on without end. This is referred to as *continuity*. A computer's number system cannot be continuous. It stores numbers using a fixed string of digits, which means it can only represent finitely many values. Real numbers have to be rounded to the nearest representable value.

The real numbers stretch infinitely in both directions. There is no largest real number and no smallest positive real number. A computer's memory is finite, so its number system must have both a largest and a smallest representable value. Numbers too large to

store cause a problem called *overflow*. Numbers too small to distinguish from zero cause *underflow*.

Exercise 1 Overflow and Underflow

Suppose a floating-point system can only represent numbers between 10^{-4} and 10^{8}.

1. Planck's constant is approximately 6.63×10^{-34} J·s. Does it cause underflow, overflow, or neither in this system?
2. The speed of light is approximately 3.00×10^{8} m/s. Does it cause underflow, overflow, or neither in this system?
3. Give one example of a number that would cause overflow.

Working Space

Answer on Page 9

So what can we do?

We cannot fix these limitations, because they are baked into the nature of finite computation. What we can do is design the floating-point system carefully so that the errors it introduces are small, predictable, and manageable. That is exactly what engineers and mathematicians have spent decades working out, and it is what the rest of this chapter is about!

The key insight is that a floating-point number is not just a string of digits. A floating-point number has three parts: a *sign*, a *significand* (sometimes called the mantissa), and an *exponent*.

1.2 Binary and the Floating-Point System

In the previous section you saw that computers cannot store most numbers exactly. Now the natural question is: how do they store numbers at all? The answer starts at the hardware level, with the smallest possible piece of information a computer can hold.

1.2.1 Binary System

Every process inside a CPU comes down to the flow of electrical current through tiny switches called *transistors*. A transistor is either on (carrying current) or off (not carrying current). That gives us exactly two states, which we write as 0 and 1. This is why computers use *binary*: it is the only number system that hardware can physically implement with a simple on/off switch.

A single binary digit is called a *bit*. Eight bits grouped together form a *byte*. You have probably seen units like megabyte (MB) or gigabyte (GB) before. One megabyte is 10^6 bytes, and one gigabyte is 10^9 bytes. A modern smartphone holds tens of billions of bits.

Exercise 2 Bits and bytes

Working Space

1. A file is 4 megabytes. How many bytes is that? How many bits?
2. A hard drive stores 500 gigabytes. How many bits can it hold? Write your answer using scientific notation.
3. A single bit can be 0 or 1. Two bits together can be 00, 01, 10, or 11, which are four possible values. How many distinct values can 8 bits (one byte) represent? How about 64 bits?

Answer on Page 9

1.2.2 Floating-Point Number

Recall that a floating-point number has three parts: a *sign*, a *significand* (sometimes called the mantissa), and an *exponent*.

Before we write any formulas, let us see why all three parts are necessary with a base-10 analogy.

Consider the number -0.001234. Written in scientific notation it is -1.234×10^{-3}. Three pieces of information do all the work:

- The **sign** $(-)$ tells us the number is negative.
- The **significand** (1.234) holds the meaningful digits.
- The **exponent** (-3) tells us how large or small the number is by shifting the decimal point.

Now watch what the exponent does when we fix the significand at 0.1234 and vary it in base 10:

Float	Value
0.1234×10^{-2}	0.001234
0.1234×10^{1}	1.234
0.1234×10^{4}	1234
0.1234×10^{8}	12,340,000

The decimal point *floats*, which means it can sit anywhere, controlled entirely by the exponent. That is why these are called floating-point numbers. The significand always carries the same digits; only the scale changes.

Exercise 3 The floating point in action

The significand is fixed at 0.5050 and the base is 10. Fill in the value represented by each float.

Float	Value
0.5050×10^{0}	
0.5050×10^{2}	
0.5050×10^{-1}	
0.5050×10^{5}	

Now answer: if you want to represent 505,000,000 using the same significand, what exponent do you need?

Working Space

Answer on Page 9

1.2.3 Floating-Point System

A floating-point system is defined by four numbers, written (β, p, m, M). Every float in that system looks like this:

$$\pm\,(0.d_1\,d_2\,\cdots\,d_p)_\beta\ \times\ \beta^e$$

This looks intimidating at first, but each symbol has a simple job. Let us unpack them one at a time.

β represents the *base*. This is the number that the exponent is a power of. In everyday life we use $\beta = 10$. Computers use $\beta = 2$ (binary).

p represents the *precision*. This is how many digits the significand gets to use. The significand $0.d_1d_2\cdots d_p$ has exactly p digits, each chosen from $\{0, 1, \ldots, \beta - 1\}$. More digits means more precision, that is, you can represent numbers more accurately. Fewer digits means faster arithmetic but more rounding error.

e represents the *exponent*. This shifts the significand left or right. The exponent is not free to be any integer; it is capped between a minimum m and a maximum M. Those bounds set the range of the system, i.e., how large and how small the representable numbers can be.

m and M represents the *exponent bounds*. The exponent must satisfy $m \le e \le M$. Together they control overflow and underflow. If a result needs an exponent below m, underflow occurs. If it needs an exponent above M, overflow occurs.

We say the system is *parametrized* by (β, p, m, M). Changing any one of the four gives a new floating-point system with different capabilities and different limitations.

Exercise 4 Reading the parameters

A toy floating-point system has parameters $(\beta, p, m, M) = (10, 4, -3, 4)$.

1. What base does this system use?
2. How many significant digits does it carry?
3. What are the smallest and largest allowed exponents?
4. Can this system represent 12,345? Try writing 12,345 in the form $0.d_1d_2d_3d_4 \times 10^e$ and check whether the exponent is within bounds.
5. Can it represent 0.00001? Show your reasoning.
6. Write two numbers that *can* be represented exactly, and one that causes overflow.

Working Space

Answer on Page 9

1.2.4 Connecting to Binary Floating-Point

All of the above applies to a computer's binary floating-point system, with $\beta = 2$ instead of $\beta = 10$. The significand digits $d_1, d_2, \ldots, d_p$ are now bits: each one is either 0 or 1. The exponent is a power of 2.

So a binary float looks like:

$$\pm\,(0.d_1\,d_2\,\cdots\,d_p)_2 \;\times\; 2^e, \qquad d_i \in \{0, 1\}, \quad m \le e \le M$$

The specific choices of p, m, and M define how much memory is used and how accurately numbers are stored. The international standard that fixes these choices for every modern computer is called *IEEE 754*, and it is the subject of the next section.

Appendix A

Answers to Exercises

Answer to Exercise 1 (on page 2)

1. Underflow. 6.63×10^{-34} is far below 10^{-4}.
2. Neither. 3.00×10^{8} is exactly at the upper boundary.
3. Any number greater than 10^{8}.

Answer to Exercise 2 (on page 3)

1. 4×10^{6} bytes; $4 \times 10^{6} \times 8 = 3.2 \times 10^{7}$ bits.
2. $500 \times 10^{9} \times 8 = 4 \times 10^{12}$ bits.
3. $2^{8} = 256$ values for one byte; $2^{64} \approx 1.8 \times 10^{19}$ values for 64 bits.

Answer to Exercise 3 (on page 4)

- $0.5050 \times 10^{0} = 0.5050$
- $0.5050 \times 10^{2} = 50.50$
- $0.5050 \times 10^{-1} = 0.05050$
- $0.5050 \times 10^{5} = 50{,}500$

For $505{,}000{,}000 = 0.5050 \times 10^{9}$, the exponent needed is 9.

Answer to Exercise 4 (on page 6)

1. Base 10.

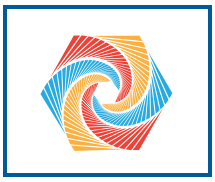
2. 4 significant digits.

3. Smallest: $m = -3$; largest: $M = 4$.

4. $12{,}345 = 0.12345 \times 10^5$. The exponent 5 exceeds $M = 4$, so this causes **overflow**. The system would need to round to $0.1235 \times 10^5 = 12{,}350$ and even then the exponent is out of range.

5. $0.00001 = 0.1000 \times 10^{-4}$. The exponent -4 is below $m = -3$, so this causes **underflow**.

6. Any number in the range 10^{-3} to $(1 - 10^{-4}) \times 10^4$ works, for example $0.5000 \times 10^2 = 50.00$ or $0.9999 \times 10^4 = 9999$. Overflow example: any number with magnitude above 9999, such as 100,000.



Index